Distributed Systems - Programming Assignment 3

Darshan Vijayaraghavan *Umar Ahmed Thameem Ahmed*

Objective

The goal of this assignment is to make the PA2 marketplace system more reliable and scalable by replicating its components. We built a rotating sequencer atomic broadcast protocol from scratch and used it to keep five copies of the customer database in sync. For the product database, we used Raft to replicate it across five servers. We also ran four copies each of the buyer and seller REST servers so clients can automatically switch to another if one goes down. The whole system runs in Docker and was deployed on GCP. We measured how latency and throughput change across three levels of load and four failure situations.

Overview

We implemented PA3 by extending our PA2 system (Python, FastAPI, gRPC, MySQL) with full server-side replication. The communication protocols remain unchanged: clients talk to REST servers over HTTP, REST servers communicate with the database layer over gRPC, and purchase transactions go through the SOAP-based financial service.

For customer database replication, we designed and implemented a rotating sequencer atomic broadcast protocol using UDP as the transport. When a write request arrives at any of the five customer gRPC nodes, the receiving node broadcasts a Request message (containing a unique <sender_id, local_seq_num> request ID and the client payload) to all peers. The node responsible for global sequence number k, determined by k mod n, selects a pending unordered request, assigns it sequence number k, and broadcasts a Sequence message to all nodes. Replicas deliver requests strictly in global sequence order only after confirming a majority of nodes have received both the Request and its Sequence message. Missing messages are detected and recovered via NACK-based retransmission.

The product database uses Raft for replication via the pysyncobj library across five nodes. All writes are routed through the current Raft leader, which replicates them to followers before committing. Reads are served directly from each node's local MySQL instance.

The four buyer REST servers and four seller REST servers are stateless and require no coordination protocol. Clients are pre-configured with the addresses of all five replicas and round-robin across them, automatically failing over to the next available replica if one becomes unreachable.

The entire stack is containerized using Docker Compose with a single MySQL instance hosting ten schemas (customer_db_0 - customer_db_4 and product_db_0 - product_db_4). For GCP deployment, the system was spread across three VMs: a database VM running MySQL, a gRPC VM running all fifteen gRPC replicas (five product Raft nodes, five seller nodes, five buyer nodes), and a REST VM running the eight REST replicas and the financial SOAP service.

Setup

For evaluation, we used the same Python simulation scripts as in PA2 (experiment_setup/simulate_buyer.py and experiment_setup/simulate_seller.py). Each script spawns N concurrent threads, one per client. Every client registers a unique account, logs in, then waits at a threading barrier until all clients are ready before starting operations simultaneously. Each client performs 1000 API calls for a single operation; latency is recorded per call and throughput is computed over the total run. Each experiment is repeated 10 times and averages are reported.

The four measured operations are the same as PA2:

- Buyer: get_seller_rating, search_items
- Seller: display_items_for_sale, register_item_for_sale

PA3 introduces four failure conditions evaluated at each concurrency level:

1. No failure: All replicas running normally
2. REST replica failure: One buyer REST replica and one seller REST replica are terminated mid-run and clients automatically reconnect to another replica
3. Non-leader Raft failure: One product database follower replica is terminated during the run
4. Raft leader failure: The current Raft leader for the product database is terminated and the remaining nodes elect a new leader and resume operation

Failures are injected by terminating the relevant Docker container during an active run. The latency reported for failure scenarios includes any additional delay from failover or Raft re-election.

For GCP deployment, all server components were run on three VMs: the database VM hosted MySQL, the gRPC VM hosted all Raft, seller, and buyer gRPC replicas along with their UDP broadcast listeners, and the REST VM hosted all buyer and seller REST replicas plus the financial SOAP service. Client load generators ran from a separate machine external to the GCP VPC, communicating with the REST layer over the public network. Firewall rules were configured to allow public access to the REST and financial service ports while keeping gRPC and MySQL traffic internal to the VPC.

Experiment 1: Average response time for each client function when all replicas run normally (no failures).

Scenario 1: 1 buyer + 1 seller respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
41.88	23.892	46.263	21.635

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
88	11.36	226	4.42

Scenario 2: 10 buyers + 10 sellers respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
103.782	96.36	136.13	73.49

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
629.57	15.88	7844.594	1.277

Scenario 3: 100 buyers + 100 sellers respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
884.62	113.38	1143.65	101.2

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
8347.41	17.28	11364.59	9.24

Experiment 2: Average response time for each client function when one server-side sellers interface replica and one server-side buyers interface to which some of the clients are connected fail.

Scenario 1: 1 buyer + 1 seller respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
65.83333333	15.19333333	68.19	14.66666667

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
764.0666667	1.31	939.5733333	1.063333333

Scenario 2: 10 buyers + 10 sellers respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
195.7366667	51.17333333	242.1866667	41.37

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
7404.796667	1.35	8631.346667	1.16

Scenario 3: 100 buyers + 100 sellers respectively

get_seller_rating		search_items	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
1406.893333	71.93333333	1692.38	59.82

display_items_for_sale		register_item_for_sale	
Latency (ms)	Throughput (ops/sec)	Latency (ms)	Throughput (ops/sec)
8705.25	4	19412.67	7.62

Experiment 3: Average response time for each client function when one product database replica (not the leader) fails.

Scenario 1: 1 buyer + 1 seller respectively

search_items	
Latency (ms)	Throughput (ops/sec)
46.43	21.54

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
131.925	7.585

Scenario 2: 10 buyers + 10 sellers respectively

search_items	
Latency (ms)	Throughput (ops/sec)
141.75	70.94333333

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
714.0333333	14.00333333

Scenario 3: 100 buyers + 100 sellers respectively

search_items	
Latency (ms)	Throughput (ops/sec)
965.91	103.65

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
8345.663333	16.47333333

Experiment 4: Average response time for each client function when the product database replica acting as leader fails.

Scenario 1: 1 buyer + 1 seller respectively

search_items	
Latency (ms)	Throughput (ops/sec)
49.34	20.27

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
246.17	4.06

Scenario 2: 10 buyers + 10 sellers respectively

search_items	
Latency (ms)	Throughput (ops/sec)
171.51	56.9

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
919.7	10.87

Scenario 3: 100 buyers + 100 sellers respectively

search_items	
Latency (ms)	Throughput (ops/sec)
1040.55	96.22

display_items_for_sale	
Latency (ms)	Throughput (ops/sec)
18101.31	6.94

Performance Analysis

Scenario 1: Single Client Performance (1 Seller + 1 Buyer)

In Scenario 1, with just one buyer and one seller running, the baseline measurements show the best performance across all functions. All replicas are healthy and requests complete without any failover or recovery overhead. When one buyer REST replica and one seller REST replica fail, latency goes up noticeably. This is expected because the client first tries to reach the failed server, waits for the connection to time out, and then reconnects to another available replica. That reconnection and retry cycle adds directly to the response time, and we can see this in both buyer-side and seller-side operations like get_seller_rating and display_items_for_sale.

For the database failure cases, losing a follower replica causes some degradation, but less than the REST failure case. Since the follower was not responsible for making decisions, the remaining nodes can continue operating without any client-side reconnection overhead. When the leader fails, however, the impact is greater. The remaining nodes detect that heartbeats from the leader have stopped, wait out an election timeout, and then one node triggers a vote. Once a candidate wins a majority, it becomes the new leader and must first apply any uncommitted log entries before accepting new writes. This entire election and recovery process introduces extra latency that a follower failure simply does not.

The result is a clear trend from best to worst: baseline < follower failure < leader failure < REST failure. For display_items_for_sale, the pattern is: 88ms (baseline) < 131.925ms (follower failure) < 246.17ms (leader failure) < 764.06ms (REST failure). The same ordering holds across all buyer and seller operations.

Write-heavy operations like register_item_for_sale are consistently slower than read-heavy ones throughout. Writes require coordination and acknowledgement across replicas before the server can respond, while reads involve less synchronization overhead. Overall, Scenario 1 shows the expected fault-tolerance trade-off, meaning, the system stays functional under failures, but at the cost of higher latency, with REST failures being the most expensive and leader failures more costly than follower failures.

Scenario 2: Moderate Concurrency (10 Sellers + 10 Buyers)

With 10 concurrent buyers and 10 concurrent sellers, we see the same performance trend as Scenario 1, baseline is fastest, followed by follower failure, then leader failure, then REST failure, but the gaps between these cases grow significantly under load.

The REST layer becomes a clear bottleneck. When a REST replica fails, multiple clients retry at the same time, causing delays to stack on top of each other. For get_seller_rating, latency increases from about 103.78ms at baseline to about 195.73ms under REST failure. The effect is much more pronounced on the seller side, display_items_for_sale jumps from about 629ms at baseline to about 7404ms under REST failure, showing how retry overhead compounds when many clients experience it simultaneously.

Write operations degrade more than reads here too, just as in Scenario 1, but the effect is larger. Write-heavy functions already reach about 7844ms even at baseline, and worsen further under failures, reflecting the sensitivity of write paths to coordination and replication overhead. Read-heavy functions like search_items hold up comparatively better.

It is also worth noting the infrastructure context. The system runs in Docker containers, and at this level of concurrency, tens of clients each issuing thousands of requests, resource contention across CPU, memory, network, and container scheduling becomes a real contributor to latency. This background pressure is most visible in failure scenarios where retries, timeouts, and reconnections add further stress on top of an already loaded system.

Overall, Scenario 2 shows that concurrency amplifies the existing bottlenecks, particularly in the REST layer and write-heavy paths.

Scenario 3: High Concurrency (100 Sellers + 100 Buyers)

At 100 concurrent buyers and 100 concurrent sellers, the system clearly hits its limits. Even the baseline performance degrades heavily, seller-side functions like display_items_for_sale reach around 8347ms and write operations like register_item_for_sale go up to around 11364ms. Compared to Scenario 2, this is roughly a 10–12x increase, showing the system does not scale linearly under high concurrency. On the buyer side, get_seller_rating shows around an 8x increase and search_items around a 10x increase. Even read-heavy operations begin to suffer once the system reaches resource saturation. At this point, queuing delays, thread contention, and resource exhaustion dominate over network overhead alone.

The same general pattern holds: baseline < follower failure < leader failure < REST failure. But leader failure becomes noticeably more damaging at this scale. For example, display_items_for_sale goes from around 8347ms at baseline to around 18101ms under leader failure. This is because with a large number of concurrent write requests in flight when the leader goes down, the election timeout, voting process, and the new leader's catch-up phase all happen against a growing backlog. Once the new leader is confirmed and starts accepting writes, it has to work through everything that queued up during the disruption, which drives latency up significantly. A follower failure avoids all of this, which is why the gap between follower failure and leader failure is wider here than in earlier scenarios.

Buyer-side APIs hold up slightly better than seller-side APIs across all failure conditions. The difference between baseline and follower failure stays relatively small for buyers, while seller write operations suffer more severely. This reflects the fact that read-heavy operations degrade more gradually, whereas write-heavy paths are far more sensitive to coordination overhead, replication delays, and system contention under heavy load. Overall, Scenario 3 confirms that the system has reached its scalability limits, where infrastructure constraints and distributed coordination overhead together dominate performance.

Comparison Across Scenarios

Across all three scenarios, latency grows consistently as concurrency increases. Taking the baseline seller operations as an example, latency goes from about 88ms in Scenario 1 to about 629ms in Scenario 2, and then to about 8347ms in Scenario 3. That is roughly a 7x jump from Scenario 1 to 2, and a further 13x jump from Scenario 2 to 3. This non-linear growth makes it clear that the system does not scale efficiently, as more clients pile on, bottlenecks like resource contention, coordination overhead, and queuing delays take over and dominate the response time. The same pattern holds consistently across both buyer and seller functions.

Throughput does increase as we go from 1 to 10 to 100 clients, since more clients are issuing requests in parallel. But this comes at a steep cost. Each step up in throughput brings a disproportionately larger increase in latency. By Scenario 3, the gains in total throughput are far outweighed by how much individual requests slow down. This reflects the classic trade-off in distributed systems: higher concurrency can improve overall throughput, but once the system starts approaching its limits, latency suffers dramatically and the efficiency of each additional client drops sharply.

PA2 vs PA3 Performance Comparison

Comparing PA3 with PA2 shows clear performance trade-offs introduced by adding replication across all components:

Single Client: For buyer operations, PA3 is only slightly slower than PA2. get_seller_rating averages 36ms in PA2 versus 42ms in PA3, about a 15% increase, and search_items goes from 37ms to 46ms, roughly a 25% increase. These small differences come from the extra network hops through replicated gRPC nodes. Seller operations, however, show a much larger gap. display_items_for_sale averages 66ms in PA2 but 343ms in PA3, a 5x increase. register_item_for_sale goes from 68ms in PA2 to 614ms in PA3, nearly a 9x increase. Even with just one client, every write in PA3 has to go through a full Raft consensus round before the server responds. The leader must replicate the entry to a majority of followers and wait for their acknowledgement before committing. This alone adds hundreds of milliseconds that simply do not exist in PA2's direct single-node writes.

Moderate Concurrency (10 Clients): For buyer operations, PA2 achieves 82ms and 122 ops/sec for get_seller_rating, while PA3 shows 155ms and 68 ops/sec, latency roughly doubled and throughput nearly halved. For seller writes, the gap becomes much larger. PA2 averages 273ms and 37 ops/sec for display_items_for_sale, while PA3 shows 5104ms and 1.97 ops/sec, about a 19x increase in latency and a 95% drop in throughput. The core issue is that Raft serializes all writes through a single leader. With 10 sellers all trying to write at the same time, they queue up waiting for the leader to replicate each write to a majority before accepting the next one. PA2 had no such constraint: each write went straight to one database without coordination. This is where replication adds the most visible cost.

High Concurrency (100 Clients): For buyer operations, PA2 holds up reasonably well, get_seller_rating at 765ms and 110 ops/sec. PA3 shows 32589ms and 4.3 ops/sec, a 42x increase in latency. search_items is even worse at nearly 48 seconds versus PA2's 831ms, a 57x increase. The atomic broadcast protocol cannot keep up with 100 concurrent buyers. Because global ordering is assigned by a single rotating sequencer at a time, and requests must be delivered strictly in order, the queue grows much faster than it drains. PA2 had none of this ordering constraint.

For seller operations at 100 clients, the numbers appear to reverse, PA3 shows 885ms for display_items_for_sale versus PA2's 2535ms, and 1144ms for register_item_for_sale versus PA2's 1841ms. However, this should not be taken as evidence that PA3 scales better for writes. Raft still routes all writes through a single leader regardless of the number of clients, so no genuine parallelism is gained. The more likely explanations are batching effects under heavy load, pipeline saturation, or differences in system conditions between the two measurement runs. These numbers should be treated with caution.

Architectural Trade-offs: PA2 is faster because it has no coordination overhead. Each request goes straight to one database and comes back. PA3 adds cost at every step, meaning, UDP broadcasts and global ordering for customer database writes, Raft consensus rounds for product database writes, and more network hops through replicated gRPC nodes. This overhead is most painful under high write concurrency, where the consensus protocols become the dominant bottleneck.

However, what PA3 gains in exchange is something PA2 never had fault tolerance. In PA2, if any single component fails, the system goes down. In PA3, individual REST replicas, gRPC nodes, and even the Raft leader can fail and the system keeps running. The customer database stays consistent across five nodes over unreliable UDP. The product database survives node crashes through Raft leader election. The REST layer scales horizontally with no code changes. For most real-world systems where reliability and uptime matter, this trade-off of higher latency in exchange for resilience is the right one to make.